

Ingeniería de Software II

Teoría y Laboratorio 2026.2

Proyecto de clase 2026.2

Sistema para la Gestión, Validación y Administración de un Banco de Preguntas para la Preparación de las Pruebas Saber Pro

Fecha de creación: Julio 28 de 2026
Última modificación: Agosto 3 de 2026

[Descripción](#)

[Planteamiento del problema](#)

[Objetivo general](#)

[Requisitos](#)

[Requerimientos funcionales](#)

[HU01. Gestión de usuarios del sistema](#)

[HU02. Gestión del banco de preguntas](#)

[HU03. Validación estructural](#)

[HU04. Gestión del ciclo de vida](#)

[HU05. Revisión por pares](#)

[HU06. Simulacros](#)

[HU07. Seguimiento académico](#)

[HU08. Administración](#)

[Requerimientos no funcionales](#)

[Usabilidad](#)

[Rendimiento](#)

[Seguridad](#)

[Modificabilidad](#)

[Confiabilidad](#)

[Escalabilidad](#)

[Restricciones](#)

[1. Tecnologías de desarrollo](#)

[2. Restricciones de alcance académico](#)

[Trabajo en equipo](#)

[Entregables](#)

Descripción

El proyecto consiste en desarrollar una plataforma que permita gestionar el ciclo de vida de preguntas de selección múltiple con única respuesta, destinadas a la preparación de estudiantes del programa de Ingeniería de Sistemas para las pruebas Saber Pro. El sistema deberá facilitar la creación, validación, revisión por pares, aprobación, publicación y utilización de preguntas en simulacros de evaluación, garantizando el cumplimiento de criterios de calidad pedagógica, técnica y estructural.

La plataforma contará con diferentes perfiles de usuario (administradores, autores, revisores, docentes y estudiantes) y ofrecerá herramientas para la construcción colaborativa de un banco de preguntas, la generación de simulacros, el seguimiento del desempeño de los estudiantes y la obtención de estadísticas que apoyen los procesos de mejoramiento académico.

Planteamiento del problema

Las pruebas Saber Pro constituyen uno de los principales mecanismos para evaluar las competencias adquiridas por los estudiantes de educación superior en Colombia. En consecuencia, las instituciones universitarias desarrollan estrategias para fortalecer la preparación de sus estudiantes mediante simulacros, bancos de preguntas y actividades de entrenamiento.

Sin embargo, la construcción de un banco de preguntas de calidad representa un proceso complejo. En muchas ocasiones, las preguntas son elaboradas de forma aislada por diferentes docentes, sin seguir criterios homogéneos de redacción, estructura o validación. Esto ocasiona problemas como preguntas ambiguas, distractores poco plausibles, inconsistencias entre el contexto y la pregunta, errores conceptuales y ausencia de procesos de revisión que garanticen la calidad de los ítems.

Adicionalmente, la mayoría de herramientas disponibles se enfocan únicamente en almacenar preguntas y presentar cuestionarios, dejando de lado aspectos fundamentales como la administración del ciclo de vida de una pregunta, la revisión por pares, la trazabilidad de las modificaciones, la aplicación de criterios de calidad, el seguimiento del desempeño de los estudiantes y la obtención de indicadores que apoyen el mejoramiento continuo del banco de preguntas.

Por esta razón, surge la necesidad de desarrollar un sistema de información que permita gestionar integralmente el proceso de creación, validación, administración y utilización de preguntas para la preparación de las pruebas Saber Pro, incorporando mecanismos que aseguren la calidad de los ítems y faciliten su reutilización en simulacros de evaluación.

Objetivo general

Desarrollar un sistema web para la gestión integral de un banco de preguntas orientado a la preparación

de estudiantes para las pruebas Saber Pro, permitiendo administrar el ciclo de vida de las preguntas, garantizar su calidad mediante procesos de revisión por pares y facilitar la generación de simulacros de evaluación.

Requisitos

Requerimientos funcionales

HU01. Gestión de usuarios del sistema

RF-01. El sistema deberá permitir registrar usuarios que interactúan con la aplicación.

RF-02. El sistema deberá permitir autenticación mediante usuario y contraseña.

RF-03. El sistema deberá administrar diferentes roles:

- Administrador
- Autor de preguntas
- Revisor
- Docente
- Estudiante

HU02. Gestión del banco de preguntas

RF-04. El sistema deberá permitir crear preguntas.

RF-05. Cada pregunta deberá estar conformada por:

- Contexto
- Pregunta directa
- Cuatro distractores
- Respuesta correcta
- Justificación de la respuesta
- Bibliografía
- Competencia
- Tema
- Subtema
- Nivel de dificultad

RF-06. El sistema deberá permitir modificar preguntas mientras se encuentren en estado "Borrador".

RF-07. El sistema deberá permitir consultar preguntas mediante filtros.

HU03. Validación estructural

RF-08. El sistema deberá validar automáticamente que toda pregunta contenga un contexto.

RF-09. El sistema deberá validar que exista una única pregunta directa.

RF-10. El sistema deberá verificar que existan exactamente cuatro distractores.

RF-11. El sistema deberá validar que exista únicamente una respuesta correcta.

RF-12. El sistema deberá impedir el uso de expresiones como:

- Todas las anteriores
- Ninguna de las anteriores

RF-13. El sistema deberá verificar que los distractores respeten criterios básicos de longitud y estructura gramatical.

HU04. Gestión del ciclo de vida

RF-14. El sistema deberá administrar los siguientes estados de una pregunta:

- Borrador
- Pendiente de revisión
- En revisión
- Aprobada
- Rechazada
- Publicada
- Archivada

RF-15. El sistema deberá controlar las transiciones válidas entre estados.

HU05. Revisión por pares

RF-16. El sistema deberá permitir asignar revisores a una pregunta.

RF-17. Cada revisor deberá diligenciar un formato de evaluación.

RF-18. El sistema deberá registrar observaciones realizadas por los revisores.

RF-19. El sistema deberá almacenar el historial de revisiones.

RF-20. El sistema deberá permitir aprobar o rechazar una pregunta.

HU06. Simulacros

RF-21. El sistema deberá permitir generar simulacros utilizando preguntas publicadas.

RF-22. El sistema deberá generar simulacros aleatorios de acuerdo con competencias, temas o niveles de dificultad.

RF-23. El sistema deberá controlar el tiempo máximo del simulacro.

RF-24. El sistema deberá calificar automáticamente las respuestas.

HU07. Seguimiento académico

RF-25. El sistema deberá almacenar el historial de simulacros realizados por cada estudiante.

RF-26. El sistema deberá generar estadísticas individuales.

RF-27. El sistema deberá identificar fortalezas y debilidades por competencia.

RF-28. El sistema deberá generar reportes para docentes y coordinadores.

HU08. Administración

RF-29. El sistema deberá mantener la trazabilidad de todas las modificaciones realizadas a una pregunta.

RF-30. El sistema deberá registrar la fecha y el usuario responsable de cada modificación.

Requerimientos no funcionales

Usabilidad

RNF-01. La interfaz deberá ser intuitiva y consistente.

RNF-02. El sistema deberá ser accesible desde navegadores web modernos.

RNF-03. La curva de aprendizaje para un usuario nuevo no deberá superar treinta minutos.

Rendimiento

RNF-04. El sistema deberá responder una consulta en menos de tres segundos bajo condiciones normales.

RNF-05. El sistema deberá soportar al menos cien usuarios concurrentes.

Seguridad

RNF-06. El sistema deberá implementar autenticación de usuarios.

RNF-07. El sistema deberá restringir el acceso según el rol del usuario.

RNF-08. Todas las contraseñas deberán almacenarse utilizando algoritmos de hash seguros.

RNF-09. El sistema deberá registrar los eventos relevantes de auditoría, como inicio de sesión, cambios de estado de las preguntas y acciones de revisión.

Modificabilidad

RNF-10. El código fuente deberá seguir principios SOLID y buenas prácticas de diseño orientado a objetos.

RNF-11. La arquitectura deberá facilitar el mantenimiento y la incorporación de nuevos módulos.

RNF-12. El sistema deberá mantener separación entre la lógica de negocio, la interfaz de usuario y la persistencia.

RNF-13. La arquitectura deberá permitir incorporar nuevos tipos de preguntas, nuevos criterios de revisión, nuevas competencias y nuevos módulos sin requerir cambios significativos en los componentes existentes.

RNF-14. El sistema deberá favorecer un bajo acoplamiento y una alta cohesión entre sus componentes, facilitando su mantenimiento y evolución.

Confiabilidad

RNF-15. El sistema deberá garantizar la integridad de la información almacenada.

RNF-16. Ninguna pregunta podrá eliminarse físicamente; únicamente podrá cambiar a un estado de inactiva o archivada, preservando la trazabilidad del banco de preguntas.

Escalabilidad

RNF-17. El sistema deberá soportar al menos **500 usuarios concurrentes** manteniendo un tiempo promedio de respuesta inferior a **3 segundos** para las operaciones de consulta. [La escalabilidad habla del crecimiento desde hasta, por ejemplo de 100 usuarios concurrente](#)

RNF-18. La arquitectura deberá permitir incrementar la capacidad del sistema mediante la incorporación de nuevos recursos de hardware o instancias de ejecución sin requerir modificaciones en la lógica de negocio.

Restricciones

1. Tecnologías de desarrollo

Se sugiere trabajar en lenguaje Java y la interfaz gráfica se hará en Swing o JavaFx. El motor de base de datos se sugiere MariaDb o Postgres.

La elección tecnológica deberá ser **argumentada técnicamente** por cada equipo, indicando ventajas, desventajas y razones de selección.

2. Restricciones de alcance académico

La solución desarrollada debe ajustarse al **contexto académico del curso**, por lo cual:

- No es obligatorio implementar integraciones externas reales (por ejemplo, WhatsApp, SMS, Envío de Emails)
- Las funcionalidades pueden implementarse mediante **simulación o prototipos funcionales**, siempre que se justifique su diseño.

Trabajo en equipo

Se debe trabajar en grupos de **tres a cinco personas**. Es importante que cada equipo trabaje de forma colaborativa, donde haya buenas relaciones entre todos sus integrantes, tareas individuales y metas comunes. El equipo debe buscar herramientas que favorezcan la comunicación durante el proyecto y la coordinación de tareas. Se recomiendan: Taiga.io, Trello o Jira. Durante el semestre **no se aceptan cambios de integrantes entre grupos**.

Entregables

Para los tres cortes se debe entregar:

1. Especificación de **requisitos funcionales** (de los más importantes, de los que generen valor al cliente) ya sea mediante escenarios de casos de uso, o Historias de Usuarios y criterios de aceptación. Los requisitos deben estar acompañados de prototipos de la interfaz de usuario mediante [wireframes](#) y algún método de validación de usabilidad como thinking aloud.
2. Especificación de requisitos **NO funcionales** mediante escenarios de calidad.
3. Elegir y justificar el **tipo de aplicación** a construir: escritorio, web, móvil, híbrida.
4. Elegir un **estilo arquitectónico** acorde a los requisitos funcionales y no funcionales. Se debe justificar por qué.
5. Elegir la **tecnología** (lenguaje, framework de desarrollo, motor de base de datos, etc.). En resumen, las preguntas que tenemos que responder son: ¿Qué tecnologías ayudan a implementar los estilos arquitecturales seleccionados? ¿Qué tecnologías ayudan a implementar el tipo de aplicación seleccionada? ¿Qué tecnologías ayudan a cumplir con los requisitos no funcionales especificados?
6. Documentación de la arquitectura, ya sea con el modelo C4 o UML.
7. Prototipo funcional que compile, ejecute, pase las pruebas y potencialmente sea usado por el cliente.

Estos puntos deben estar en un documento en PDF. En el punto 7, debe colocarse la URL del repositorio GitHub del proyecto. A continuación un formato para este documento:

1. Portada
2. Resumen
3. Requisitos funcionales en forma de historias épicas, historias de usuario y criterios de aceptación (Referenciar la hoja de cálculo de la plantilla de historias de usuario).
4. Prototipos y test de usabilidad (en caso de haberlos realizado)
5. Requisitos no funcionales en forma de escenarios de atributos de calidad.
6. Diagrama entidad relación
7. Arquitectura del sistema usando el modelo C4 más algunos diagramas del modelo 4+1 (los más relevantes).
8. Decisiones de arquitectura y su justificación (Selección del tipo de aplicación, Selección del estilo arquitectónico, Selección de tecnologías)
9. URL del repositorio de GitHub.

Corte 1 - Primera iteración: hacer una implementación monolítica en capas (MVC). Utilizar principios de diseño y patrones de diseño para garantizar la **modificabilidad** de la aplicación. Se debe implementar al menos un conjunto de historias de usuario de alto valor para el cliente (que serán definidas en otro

archivo).

Corte 2 - Segunda iteración: hacer una refactorización de la iteración 1 implementando una solución distribuida con arquitecturas microservicios y orientada a eventos para garantizar el la **escalabilidad**. Utilizar los patrones de diseño vistos.

Corte 3 - Tercera iteración: seguir con la implementación distribuida y utilizar una **arquitectura hexagonal**, abarcando tres requisitos funcionales nuevos de alta prioridad para el cliente. Utilizar más patrones de diseño GoF. Tener en cuenta aspectos de **seguridad** para la autenticación y la autorización.